

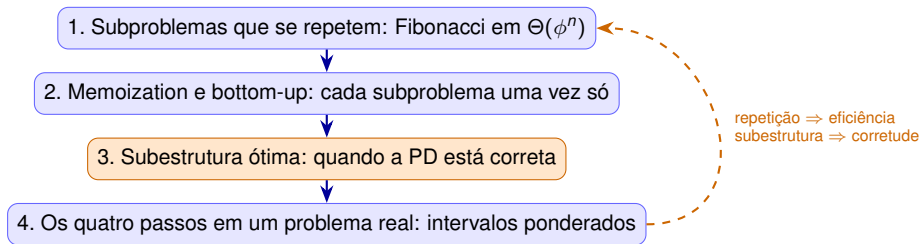
Programação dinâmica

Memoization e subestrutura ótima

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

8 de outubro de 2026



Programação dinâmica (PD) = **recursão** + **tabela**. As duas condições para usá-la são os dois assuntos centrais desta aula.

Subproblemas que se repetem

Problema:

Em divisão e conquista (Karatsuba, Strassen, par mais próximo), os subproblemas são **disjuntos**: cada metade é resolvida uma vez.

Muitas recorrências, porém, geram subproblemas que se **sobrepõem**. A recursão direta resolve o mesmo subproblema muitas vezes e o tempo vira **exponencial**.

Solução:

Resolver cada subproblema distinto **uma única vez** e guardar a resposta em uma **tabela**. Na próxima vez que ele aparecer, basta consultar.

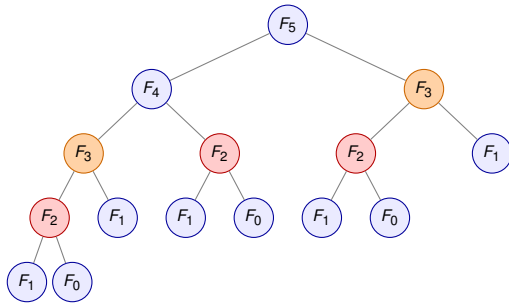
Troca-se memória por tempo: se há poucos subproblemas distintos (polinomial), o algoritmo fica polinomial.

Para aprofundar: Cormen et al. (2009), introdução do cap. 15 [2]; Levitin (2012), introdução do cap. 8 [8].

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}.$$

```
long fib( int n) {  
    if (n <= 1) return n;  
    return fib( n-1) + fib( n-2);  
}
```

A árvore ao lado é a de `fib(5)`: 15 chamadas para 6 valores distintos. F_3 é calculado **2 vezes**, F_2 **3 vezes**.



Para aprofundar: Dasgupta et al. (2008), §0.2 [4]; Levitin (2012), introdução do cap. 8 [8].

Quantas chamadas faz `fib(n)`?

Seja $C(n)$ o número de chamadas:

$$C(0) = C(1) = 1, \quad C(n) = C(n-1) + C(n-2) + 1.$$

Por indução, $C(n) = 2F_{n+1} - 1$. Como $F_n \approx \phi^n / \sqrt{5}$,
com $\phi = \frac{1+\sqrt{5}}{2} \approx 1,618$:

$$C(n) \in \Theta(\phi^n).$$

O tempo cresce **como a própria resposta**: só há 6 valores distintos em `fib(5)`, mas o algoritmo faz $2F_6 - 1 = 15$ chamadas.

n	chamadas $C(n)$	distintos
5	15	6
10	177	11
20	21 891	21
30	2 692 537	31
40	331 160 281	41
50	$\approx 4 \cdot 10^{10}$	51

A 10^9 chamadas/s, `fib(50)` leva ≈ 40 s;
`fib(70)`, mais de um dia.

Para aprofundar: Dasgupta et al. (2008), §0.2 [4]; Cormen et al. (2009), Exercícios 3.2-6 e 3.2-7 (razão áurea) [2].

Definição

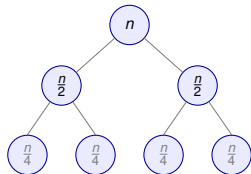
Um problema tem **subproblemas sobrepostos** quando um algoritmo recursivo para ele resolve **os mesmos subproblemas repetidamente**, em vez de gerar sempre subproblemas novos. Interessa o caso em que o número de subproblemas *distintos* é polinomial no tamanho da entrada.

Intuição

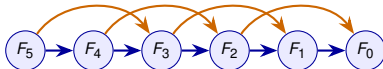
Desenhe cada subproblema *uma vez* e ligue-o aos que ele usa: em D&C sai uma **árvore**; em PD, um **DAG** pequeno.

Para aprofundar: Cormen et al. (2009), §15.3, "Overlapping subproblems" e §15.1, "Subproblem graphs" [2].

D&C: subproblemas disjuntos



PD: grafo de subproblemas de fib(5)



Por que “programação dinâmica”?

- Richard Bellman formulou o método nos anos 1950, na RAND Corporation, para problemas de decisão em vários estágios; o livro *Dynamic Programming* é de 1957.
- “Programação” tem o sentido de **planejamento** preenchendo uma **tabela** (como em “programação linear”), não de escrever código.
- Segundo a autobiografia de Bellman, o nome foi escolhido para soar bem a um secretário de Defesa avesso à palavra “pesquisa”: “dinâmica” indicava estágios e tempo, e era difícil de usar em sentido pejorativo.
- A ideia reaparece em toda a disciplina: Bellman-Ford, Floyd-Warshall, mochila, LCS e alinhamento de sequências em bioinformática.



Richard Bellman (1920–1984)

Imagem: Wikimedia Commons

Para aprofundar: Bellman (1957) [1]; Dreyfus (2002) [5]; Cormen et al. (2009), notas do cap. 15 [2].

O que aprendemos

- A recursão direta de Fibonacci faz $2F_{n+1} - 1 \in \Theta(\phi^n)$ chamadas, mas só existem $n + 1$ subproblemas distintos.
- **Subproblemas sobrepostos:** a recursão volta aos mesmos subproblemas; o grafo de subproblemas é um DAG pequeno, não uma árvore enorme.
- Divisão e conquista: subproblemas disjuntos. PD: subproblemas compartilhados, resolvidos uma vez e guardados em tabela.

Para aprofundar: Cormen et al. (2009), §15.3 [2].

Memoization e bottom-up

Problema:

A recorrência $F_n = F_{n-1} + F_{n-2}$ está **correta**; o problema é só a repetição. Reescrever o algoritmo do zero dá trabalho e pode introduzir erros.

Solução: duas formas de aproveitar a mesma recorrência.

- **Top-down com memoization:** mantém a recursão e anota cada resposta na tabela antes de devolvê-la.
- **Bottom-up:** preenche a tabela em uma ordem em que cada entrada só depende de entradas já preenchidas.

Ambas fazem $\Theta(n)$ trabalho para Fibonacci.

Para aprofundar: Cormen et al. (2009), §15.1, "Top-down with memoization" e "Bottom-up method" [2].

Definição

Um algoritmo recursivo **memoizado** guarda em uma tabela a resposta de cada subproblema na primeira vez que a calcula. Antes de calcular, consulta a tabela: se a resposta já está lá, devolve-a sem recursão.

Intuição

Um “memorando” para si mesmo: o termo *memo function* é de Michie (1968), daí *memoization*, sem “r”.

```
long memo[MAX]; /* inicializado com -1 */

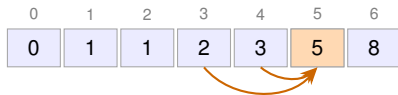
long fibMemo( int n) {
    if (memo[n] >= 0)          /* já calculado? */
        return memo[n];
    long r;
    if (n <= 1) r = n;
    else r = fibMemo( n-1) + fibMemo( n-2);
    memo[n] = r;               /* anota */
    return r;
}
```

Cada F_k é calculado uma vez: $2n - 1$ chamadas, $\Theta(n)$.

Para aprofundar: Cormen et al. (2009), §15.3, “Memoization” [2]; Michie (1968) [9].

```
long fibBU( int n) {  
    long F[MAX];  
    F[0] = 0; F[1] = 1;  
    for (int k = 2; k <= n; ++k)  
        F[k] = F[k-1] + F[k-2];  
    return F[n];  
}
```

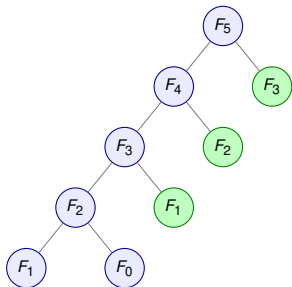
Sem recursão, sem pilha, sem teste “já calculei?”.
Como $F[k]$ só usa as duas entradas anteriores, dá
para guardar só duas variáveis: espaço $O(1)$.



ordem de preenchimento: ordem topológica inversa do DAG

	top-down memo	bottom-up
ordem subproblemas	a recursão decide só os necessários	você decide todos
custo extra	pilha + testes	nenhum

Para aprofundar: Cormen et al. (2009), §15.1, “Subproblem graphs” [2]; Dasgupta et al. (2008), §6.1 [4].



Exemplo

- A recursão desce pelo ramo esquerdo até F_1 e F_0 e anota F_2, F_3, F_4 na volta.
- Os nós **verdes** são consultas à tabela: devolvem na hora, sem filhos.
- Total: 9 chamadas (antes eram 15). Para n geral, $2n - 1$ chamadas contra $2F_{n+1} - 1$.

Para aprofundar: Cormen et al. (2009), §15.1, MEMOIZED-CUT-ROD [2]; Dasgupta et al. (2008), §6.1 [4].

O que aprendemos

- **Memoization:** mesma recursão, mais uma tabela consultada antes e preenchida depois de cada cálculo.
- **Bottom-up:** laço que preenche a tabela em ordem topológica inversa do grafo de subproblemas.
- Tempo $\approx$ (nº de subproblemas distintos) $\times$ (trabalho por subproblema). Fibonacci: $(n + 1) \times O(1) = \Theta(n)$.
- As duas versões têm o mesmo tempo assintótico; a recorrência é que precisa estar certa.

Para aprofundar: Cormen et al. (2009), §15.1 e §15.3 [2].

Subestrutura ótima

Problema:

Em Fibonacci só há *um* jeito de combinar as respostas: somar. Em problemas de **otimização** (mínimo, máximo), a recorrência escolhe a melhor entre várias opções, usando as respostas *ótimas* dos subproblemas.

Isso só é correto se a solução ótima for feita de pedaços ótimos.

Solução:

Antes de escrever a recorrência, **provar** que o problema tem **subproblemas sobrepostos** e **subestrutura ótima**, as duas condições que Cormen et al. citam para a PD funcionar.

A primeira dá a **eficiência**; a segunda, a **corretude**. É o mesmo papel do invariante de laço da Unidade 1.

Para aprofundar: Cormen et al. (2009), §15.3, "Elements of dynamic programming" [2].

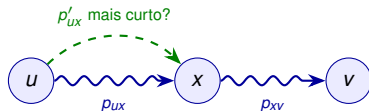
Definição

Um problema tem **subestrutura ótima** se toda solução ótima do problema contém soluções ótimas dos subproblemas que ela usa.

Intuição: o argumento “recorta e cola”

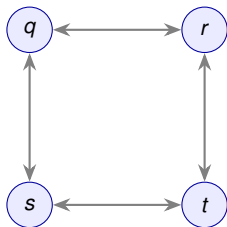
Suponha que a solução ótima usa uma solução S' *não ótima* de um subproblema. Recorte S' e cole no lugar uma solução melhor. O resultado é melhor que o ótimo: contradição.

Para aprofundar: Cormen et al. (2009), §15.3, “Optimal substructure”, e Lema 24.1 [2].



Se p'_{ux} fosse mais curto que p_{ux} ,
 $p'_{ux} p_{xv}$ seria mais curto que o caminho mínimo $u \rightsquigarrow v$.

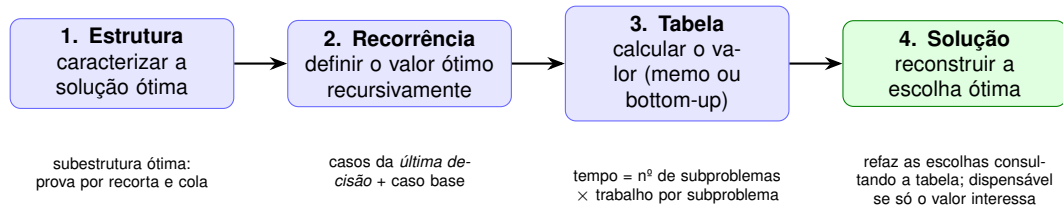
Logo, todo subcaminho de um caminho mínimo é mínimo.



- Caminho simples **mais longo** de q a t : $q \rightarrow r \rightarrow t$ (ou $q \rightarrow s \rightarrow t$), com 2 arestas.
- O subcaminho $q \rightarrow r$ **não** é o mais longo de q a r : esse é $q \rightarrow s \rightarrow t \rightarrow r$.
- O mais longo de r a t é $r \rightarrow q \rightarrow s \rightarrow t$. Juntar os dois ótimos dá $q \rightarrow s \rightarrow t \rightarrow r \rightarrow q \rightarrow s \rightarrow t$: **nem é simples**.
- Os subproblemas não são **independentes**: os vértices usados em um proíbem o outro. Sem subestrutura ótima, não há recorrência correta.

Já o caminho **mínimo** tem subestrutura ótima (frame anterior): veremos PD para ele em Warshall-Floyd.

Para aprofundar: Cormen et al. (2009), §15.3, Figura 15.6 [2].



Para aprofundar: Cormen et al. (2009), introdução do cap. 15 [2]; Kleinberg e Tardos (2006), §6.2 [7].

O que aprendemos

- **Subestrutura ótima:** a solução ótima contém soluções ótimas de subproblemas. Prova típica: recorta e cola, por contradição.
- Caminho mínimo tem; caminho simples mais longo não tem (subproblemas dependentes).
- Subproblemas sobrepostos dão eficiência; subestrutura ótima dá corretude. A PD precisa das duas.
- Roteiro: estrutura $\rightarrow$ recorrência $\rightarrow$ tabela $\rightarrow$ reconstrução.

Para aprofundar: Cormen et al. (2009), §15.3 [2].

Seleção de intervalos ponderados

Problema:

Um auditório recebe n pedidos de reserva. O pedido j ocupa o intervalo $[s_j, f_j)$ e paga v_j . Dois pedidos sobrepostos não podem ser aceitos juntos. Quais aceitar para **maximizar a receita**?

Força bruta testa os 2^n subconjuntos.

Solução:

Com todos os pesos iguais, o guloso “termina mais cedo” é ótimo (Kleinberg e Tardos, §4.1). Com pesos, **nenhum guloso natural funciona**.

Vamos aplicar os quatro passos e obter um algoritmo $O(n \log n)$.

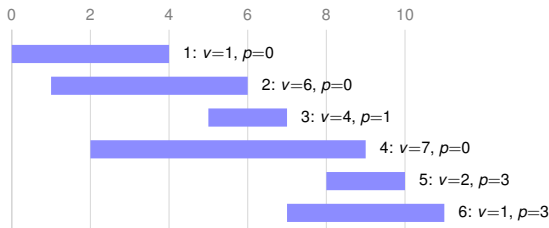
Para aprofundar: Kleinberg e Tardos (2006), §6.1 [7].

Definição

Entrada: intervalos $1, \dots, n$ ordenados por término, $f_1 \leq \dots \leq f_n$, com valores $v_j > 0$. Saída: um conjunto de intervalos **dois a dois disjuntos** com soma de valores máxima.

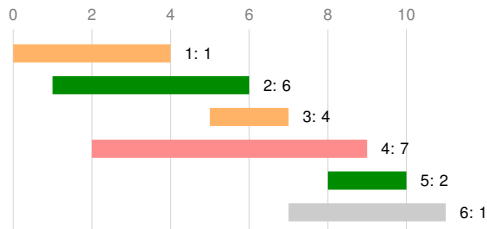
O vetor p

$p(j)$ = maior índice $i < j$ com $f_i \leq s_j$ (o último intervalo compatível com j); $p(j) = 0$ se não houver. Na instância ao lado: $p = (0, 0, 1, 0, 3, 3)$.



Para aprofundar: Kleinberg e Tardos (2006), §6.1, Figura 6.2 [7].

Exemplo: os gulosos falham nesta instância



Exemplo

critério	escolhe	valor
termina mais cedo	{1, 3, 5}	$1 + 4 + 2 = 7$
maior valor	{4}	7
ótimo	{2, 5}	$6 + 2 = 8$

Cada guloso decide olhando um intervalo de cada vez. O ótimo depende de **combinações**: é isso que a tabela vai registrar.

Para aprofundar: Kleinberg e Tardos (2006), §6.1 [7].

Seja O_j uma solução ótima para os intervalos $\{1, \dots, j\}$ e $\text{OPT}(j)$ o seu valor. Olhe para a **última decisão**: o intervalo j está ou não está em O_j .

Caso $j \notin O_j$

O_j é solução para $\{1, \dots, j-1\}$ e precisa ser **ótima** para esse subproblema; senão, troque-a por uma melhor (recorta e cola).

$$\text{OPT}(j) = \text{OPT}(j-1).$$

Caso $j \in O_j$

Os intervalos $p(j) + 1, \dots, j-1$ terminam depois de s_j e sobrepõem j : estão fora. O resto de O_j é solução **ótima** para $\{1, \dots, p(j)\}$, pelo mesmo argumento.

$$\text{OPT}(j) = v_j + \text{OPT}(p(j)).$$

Para aprofundar: Kleinberg e Tardos (2006), §6.1 [7].

Recorrência

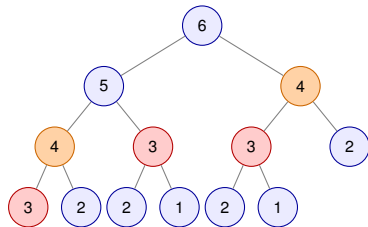
$\text{OPT}(0) = 0$ e, para $j \geq 1$,

$$\text{OPT}(j) = \max(v_j + \text{OPT}(p(j)), \text{OPT}(j-1)).$$

O intervalo j pertence a alguma solução ótima de $\{1, \dots, j\}$ se e somente se $v_j + \text{OPT}(p(j)) \geq \text{OPT}(j-1)$.

Cuidado: se $p(j) = j - 2$ para todo j , a recursão direta tem exatamente a árvore de Fibonacci: $\Theta(\phi^n)$ chamadas.

Para aprofundar: Kleinberg e Tardos (2006), §6.1, (6.1), (6.2) e Figura 6.4 [7].



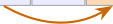
chamadas de OPT com $p(j) = j - 2$
(árvore cortada no nível 3)

Passo 3: a tabela M , memoizada ou bottom-up

```
/* M[j] == -1: ainda não calculado */  
long optMemo( int j) {  
    if (j == 0) return 0;  
    if (M[j] >= 0) return M[j];  
    long com = v[j] + optMemo( p[j]);  
    long sem = optMemo( j-1);  
    return M[j] = com >= sem ? com : sem;  
}
```

```
long optBU( int n) {  
    M[0] = 0;  
    for (int j = 1; j <= n; ++j) {  
        long com = v[j] + M[p[j]];  
        long sem = M[j-1];  
        M[j] = com >= sem ? com : sem;  
    }  
    return M[n];  
}
```

j	0	1	2	3	4	5	6
M	0	1	6	6	7	8	8



Em $j = 5$:

$\max(v_5 + M[p(5)], M[4]) = \max(2 + 6, 7) = 8$. Cada $M[j]$ é calculado uma vez com $O(1)$ trabalho: $O(n)$, dado p .

Para aprofundar: Kleinberg e Tardos (2006), §6.1, M-COMPUTE-OPT, e §6.2, ITERATIVE-COMPUTE-OPT [7].

Passo 4: reconstruindo a solução

```
/* solução ótima de {1..j} */  
void solucao( int j) {  
    while (j > 0) {  
        if (v[j] + M[p[j]] >= M[j-1]) {  
            printf( "%d ", j); /* entra */  
            j = p[j];  
        }  
        else j = j - 1;          /* fica fora */  
    }  
}
```

A tabela permite **refazer cada decisão** em $O(1)$:
 $O(n)$ no total.

Para aprofundar: Kleinberg e Tardos (2006), §6.1, FIND-SOLUTION [7].

Na instância condutora

j	teste $v_j + M[p(j)] \geq M[j-1]$	decisão
6	$1 + 6 = 7 \geq 8?$ não	$j \leftarrow 5$
5	$2 + 6 = 8 \geq 7?$ sim	5 entra , $j \leftarrow 3$
3	$4 + 1 = 5 \geq 6?$ não	$j \leftarrow 2$
2	$6 + 0 = 6 \geq 1?$ sim	2 entra , $j \leftarrow 0$

Solução $\{2, 5\}$, valor $M[6] = 8$.

etapa	tempo
ordenar por término	$O(n \log n)$
calcular $p(1), \dots, p(n)$ (busca binária em f)	$O(n \log n)$
preencher M (memo ou bottom-up)	$O(n)$
reconstruir a solução	$O(n)$
total	$O(n \log n)$

Espaço: $O(n)$ para s, f, v, p e M .

Para aprofundar: Kleinberg e Tardos (2006), §6.1, (6.4) [7].

Conta da PD

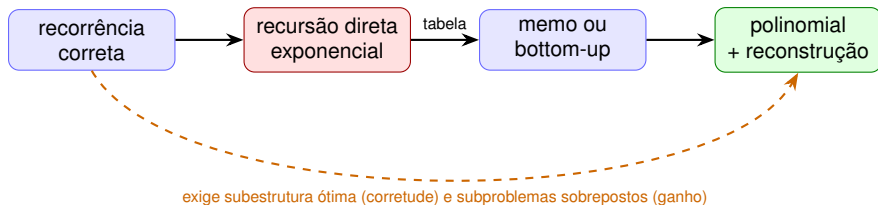
$n + 1$ subproblemas ($\text{OPT}(0), \dots, \text{OPT}(n)$) $\times$
 $O(1)$ por subproblema (duas consultas e um
 max) = $O(n)$.

Contra 2^n subconjuntos da força bruta: com
 $n = 60$, 10^{18} contra 60.

O que aprendemos

- Subproblema: $\text{OPT}(j)$, o melhor valor usando só $\{1, \dots, j\}$.
- Estrutura: j entra (sobra $\{1, \dots, p(j)\}$) ou não entra (sobra $\{1, \dots, j-1\}$); recorta e cola garante a subestrutura ótima.
- Recorrência: $\text{OPT}(j) = \max(v_j + \text{OPT}(p(j)), \text{OPT}(j-1))$, $\text{OPT}(0) = 0$.
- A recursão direta pode ser exponencial; com memo ou bottom-up, $O(n)$ após ordenar e calcular p .
- A solução é reconstruída refazendo as decisões a partir da tabela.

Para aprofundar: Kleinberg e Tardos (2006), §§6.1–6.2 [7].



Próxima aula: a mochila 0/1. O subproblema terá *dois* índices (itens e capacidade), a tabela vira uma matriz e o tempo será pseudopolinomial.

Para estudar: o corte de hastes (Cormen et al., §15.1) é outro bom exemplo dos quatro passos; a fileira de moedas (Levitin, §8.1) é o desafio do tutorial.

Para aprofundar: Cormen et al. (2009), §15.1 [2]; Levitin (2012), §§8.1–8.2 [8]; Feofiloff, *Programação dinâmica* [6].



Richard Bellman.

Dynamic Programming.

Princeton University Press, Princeton, NJ, 1957.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Introduction to Algorithms.

MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Algoritmos: Teoria e Prática.

Elsevier, Rio de Janeiro, 3 edition, 2012.

Tradução da 3. ed. norte-americana. Seção 4.5.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.

Algorithms.

McGraw-Hill, 2008.

Seção 2.1: Multiplication (o truque de Gauss).



Stuart Dreyfus.

Richard Bellman on the birth of dynamic programming.

Operations Research, 50(1):48–51, 2002.



Paulo Feofiloff.

Programação dinâmica.

Análise de Algoritmos, IME-USP.

Acesso em: 7 out. 2026.



Jon Kleinberg and Éva Tardos.

Algorithm Design.

Pearson/Addison-Wesley, 2006.

Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Donald Michie.

"memo" functions and machine learning.

Nature, 218:19–22, 1968.